
An introduction to Homotopy Type Theory and its applications

Koen Bresters
Freie Universität Berlin



Why Homotopy Type Theory?

- ▶ Invariance under isomorphism is useful: in principle want *everything* we do to be invariant under isomorphism

Why Homotopy Type Theory?

- ▶ Invariance under isomorphism is useful: in principle want *everything* we do to be invariant under isomorphism
- ▶ Consider for $n \in \mathbb{N}$ the definitions
 - ▷ an n -dimensional $\mathbb{R}$ -vector spaces
 - ▷ an n -dimensional real inner product space
 - ▷ an n -dimensional affine space
 - ▷ a smooth manifold diffeomorphic to $\mathbb{R}^n$

Why Homotopy Type Theory?

- ▶ Invariance under isomorphism is useful: in principle want *everything* we do to be invariant under isomorphism
- ▶ Consider for $n \in \mathbb{N}$ the definitions
 - ▷ an n -dimensional $\mathbb{R}$ -vector spaces
 - ▷ an n -dimensional real inner product space
 - ▷ an n -dimensional affine space
 - ▷ a smooth manifold diffeomorphic to $\mathbb{R}^n$
- ▶ All isomorphic to $\mathbb{R}^n$! $\rightsquigarrow$ Why not just work with $\mathbb{R}^n$ the whole time?

Why Homotopy Type Theory?

- ▶ Invariance under isomorphism is useful: in principle want *everything* we do to be invariant under isomorphism
- ▶ Consider for $n \in \mathbb{N}$ the definitions
 - ▷ an n -dimensional $\mathbb{R}$ -vector spaces
 - ▷ an n -dimensional real inner product space
 - ▷ an n -dimensional affine space
 - ▷ a smooth manifold diffeomorphic to $\mathbb{R}^n$
- ▶ All isomorphic to $\mathbb{R}^n$! $\rightsquigarrow$ Why not just work with $\mathbb{R}^n$ the whole time?
- ▶ A: With these definitions, 'correct' notion of isomorphism is clear, and everything we do in practice is invariant under isomorphism

Why Homotopy Type Theory?

- ▶ Invariance under isomorphism is useful: in principle want *everything* we do to be invariant under isomorphism
- ▶ Consider for $n \in \mathbb{N}$ the definitions
 - ▷ an n -dimensional $\mathbb{R}$ -vector spaces
 - ▷ an n -dimensional real inner product space
 - ▷ an n -dimensional affine space
 - ▷ a smooth manifold diffeomorphic to $\mathbb{R}^n$
- ▶ All isomorphic to $\mathbb{R}^n$! $\rightsquigarrow$ Why not just work with $\mathbb{R}^n$ the whole time?
- ▶ A: With these definitions, 'correct' notion of isomorphism is clear, and everything we do in practice is invariant under isomorphism
- ▶ Homotopy Type Theory makes this precise:
 - ▷ all mathematical objects have a notion of isomorphism generated *automatically*
 - ▷ *any* construction is invariant under isomorphism

Why type theory?

- ▶ In set theory, all objects are sets.

Why type theory?

- ▶ In set theory, all objects are sets.

Definition

A group $(G, \cdot, e)$ is *nice* if e has 3 elements.

Why type theory?

- ▶ In set theory, all objects are sets.

Definition

A group $(G, \cdot, e)$ is *nice* if e has 3 elements.

- ▶ Need to disallow looking 'inside' elements of sets.

Type theory

- ▶ basic concepts are *type judgments*:

$$"a_1 : A_1, \dots, a_n : A_n \vdash a : A"$$

- ▷ $A_1, \dots, A_n, a, A$ are placeholders for any list of symbols
- ▷ represents the idea “ a is a term of type A ” in a context given variables $a_1, a_2, \dots, a_n$

Type theory

- ▶ basic concepts are *type judgments*:

$$"a_1 : A_1, \dots, a_n : A_n \vdash a : A"$$

- ▷ $A_1, \dots, A_n, a, A$ are placeholders for any list of symbols
 - ▷ represents the idea “ a is a term of type A ” in a context given variables $a_1, a_2, \dots, a_n$
- ▶ specified *inference rules* allow us to deduce judgments from existing ones

Type theory

- ▶ basic concepts are *type judgments*:

$$"a_1 : A_1, \dots, a_n : A_n \vdash a : A"$$

- ▷ $A_1, \dots, A_n, a, A$ are placeholders for any list of symbols
 - ▷ represents the idea “ a is a term of type A ” in a context given variables $a_1, a_2, \dots, a_n$
- ▶ specified *inference rules* allow us to deduce judgments from existing ones
- ▶ Example: rules encoding the natural numbers

$$\frac{}{\Gamma \vdash \mathbb{N} : \mathbf{Type}} \quad \frac{}{\Gamma \vdash 0 : \mathbb{N}} \quad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{succ}(n) : \mathbb{N}}$$

Type theory

- ▶ basic concepts are *type judgments*:

$$"a_1 : A_1, \dots, a_n : A_n \vdash a : A"$$

- ▷ $A_1, \dots, A_n, a, A$ are placeholders for any list of symbols
 - ▷ represents the idea “ a is a term of type A ” in a context given variables $a_1, a_2, \dots, a_n$
- ▶ specified *inference rules* allow us to deduce judgments from existing ones
- ▶ Example: rules encoding the natural numbers

$$\frac{}{\Gamma \vdash \mathbb{N} : \mathbf{Type}} \quad \frac{}{\Gamma \vdash 0 : \mathbb{N}} \quad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{succ}(n) : \mathbb{N}}$$

- ▷ Example: can deduce “ $\cdot \vdash \text{succ}(\text{succ}(0)) : \mathbb{N}$ ”

Type theory

- ▶ Induction rule: to construct given $n : \mathbb{N}$ a term $c : C(n)$ suffices to
 - ▷ give a term $c_0 : C(0)$
 - ▷ given $n : \mathbb{N}$ and a term $c : C(n)$ give a term $c_s : C(\text{succ}(n))$

Type theory

- ▶ Induction rule: to construct given $n : \mathbb{N}$ a term $c : C(n)$ suffices to
 - ▷ give a term $c_0 : C(0)$
 - ▷ given $n : \mathbb{N}$ and a term $c : C(n)$ give a term $c_s : C(\text{succ}(n))$
- ▶ Type formers

name	notation	terms
function type	$A \rightarrow B$	$\lambda x.y$ for $y : B$ given $x : A$

Type theory

- ▶ Induction rule: to construct given $n : \mathbb{N}$ a term $c : C(n)$ suffices to
 - ▷ give a term $c_0 : C(0)$
 - ▷ given $n : \mathbb{N}$ and a term $c : C(n)$ give a term $c_s : C(\text{succ}(n))$
- ▶ Type formers

name	notation	terms
function type	$A \rightarrow B$	$\lambda x. y$ for $y : B$ given $x : A$
product type	$A \times B$	(a, b) for $a : A, b : B$

Type theory

- ▶ Induction rule: to construct given $n : \mathbb{N}$ a term $c : C(n)$ suffices to
 - ▷ give a term $c_0 : C(0)$
 - ▷ given $n : \mathbb{N}$ and a term $c : C(n)$ give a term $c_s : C(\text{succ}(n))$
- ▶ Type formers

name	notation	terms
function type	$A \rightarrow B$	$\lambda x.y$ for $y : B$ given $x : A$
product type	$A \times B$	(a, b) for $a : A, b : B$
dependent function type	$\prod_{a:A} B(a)$	$\lambda x.y$ for $y : B(x)$ given $x : A$

Type theory

- ▶ Induction rule: to construct given $n : \mathbb{N}$ a term $c : C(n)$ suffices to
 - ▷ give a term $c_0 : C(0)$
 - ▷ given $n : \mathbb{N}$ and a term $c : C(n)$ give a term $c_s : C(\text{succ}(n))$
- ▶ Type formers

name	notation	terms
function type	$A \rightarrow B$	$\lambda x.y$ for $y : B$ given $x : A$
product type	$A \times B$	(a, b) for $a : A, b : B$
dependent function type	$\prod_{a:A} B(a)$	$\lambda x.y$ for $y : B(x)$ given $x : A$
dependent pair type	$\sum_{a:A} B(a)$	(a, b) for $a : A, b : B(a)$

Type theory

- ▶ Induction rule: to construct given $n : \mathbb{N}$ a term $c : C(n)$ suffices to
 - ▷ give a term $c_0 : C(0)$
 - ▷ given $n : \mathbb{N}$ and a term $c : C(n)$ give a term $c_s : C(\text{succ}(n))$

- ▶ Type formers

name	notation	terms
function type	$A \rightarrow B$	$\lambda x.y$ for $y : B$ given $x : A$
product type	$A \times B$	(a, b) for $a : A, b : B$
dependent function type	$\prod_{a:A} B(a)$	$\lambda x.y$ for $y : B(x)$ given $x : A$
dependent pair type	$\sum_{a:A} B(a)$	(a, b) for $a : A, b : B(a)$

- ▶ Example:

- ▷ $\cdot \vdash \lambda(n : \mathbb{N}).\text{succ}(\text{succ}(n)) : \mathbb{N} \rightarrow \mathbb{N}$ “addition by 2 function”
- ▷ $\cdot \vdash (0, 0) : \mathbb{N} \times \mathbb{N}$

Propositional logic in type theory

- ▶ To do mathematics need propositional logic

Propositional logic in type theory

- ▶ To do mathematics need propositional logic
- ▶ Can represent propositions with types! (Curry-Howard correspondence)

proposition P		type P
giving a proof of P		giving a term $h : P$

Propositional logic in type theory

- ▶ To do mathematics need propositional logic
- ▶ Can represent propositions with types! (Curry-Howard correspondence)

proposition P	type P
giving a proof of P	giving a term $h : P$
$P \Rightarrow Q$	$P \rightarrow Q$
$P \wedge Q$	$P \times Q$
$\forall x : A. P(x)$	$\prod_{x:X} P(x)$
$\exists x : A. P(x)$	$\sum_{x:X} P(x)$

Propositional logic in type theory

- ▶ To do mathematics need propositional logic
- ▶ Can represent propositions with types! (Curry-Howard correspondence)

proposition P	type P
giving a proof of P	giving a term $h : P$
$P \Rightarrow Q$	$P \rightarrow Q$
$P \wedge Q$	$P \times Q$
$\forall x : A. P(x)$	$\prod_{x:A} P(x)$
$\exists x : A. P(x)$	$\sum_{x:A} P(x)$

- ▶ Main source of propositions: equality
 - ▷ given terms $a, b : A$ should have a type $a = b$ representing the proposition $a = b$

Identity types

► basic rules:

$$\frac{A : \mathbf{Type} \quad a : A \quad b : A}{a = b : \mathbf{Type}} \qquad \frac{A : \mathbf{Type} \quad a : A}{\text{refl}_a : a = a}$$

Identity types

- basic rules:

$$\frac{A : \mathbf{Type} \quad a : A \quad b : A}{a = b : \mathbf{Type}} \quad \frac{A : \mathbf{Type} \quad a : A}{\text{refl}_a : a = a}$$

- induction principle: given $a, b : A$ and $p : a = b$ to construct $c : C(a, b, p)$ it suffices to: given $a : A$ give $c_0 : C(a, a, \text{refl}_a)$

Identity types

- ▶ basic rules:

$$\frac{A : \mathbf{Type} \quad a : A \quad b : A}{a = b : \mathbf{Type}} \quad \frac{A : \mathbf{Type} \quad a : A}{\text{refl}_a : a = a}$$

- ▶ induction principle: given $a, b : A$ and $p : a = b$ to construct $c : C(a, b, p)$ it suffices to: given $a : A$ give $c_0 : C(a, a, \text{refl}_a)$
- ▶ can deduce basic properties:
 - ▷ $\text{inv} : (a = b) \rightarrow (b = a)$ defined by $\text{inv}(\text{refl}_a) := \text{refl}_a$
 - ▷ $\cdot : (a = b) \rightarrow (b = c) \rightarrow (a = c)$ defined by $\text{refl}_a \cdot \text{refl}_a := \text{refl}_a$
 - ▷ $\text{ap}_f : (a = b) \rightarrow f(a) = f(b)$ defined by $\text{ap}_f(\text{refl}_a) := \text{refl}_{f(a)}$

Identity types

- ▶ basic rules:

$$\frac{A : \mathbf{Type} \quad a : A \quad b : A}{a = b : \mathbf{Type}} \quad \frac{A : \mathbf{Type} \quad a : A}{\text{refl}_a : a = a}$$

- ▶ induction principle: given $a, b : A$ and $p : a = b$ to construct $c : C(a, b, p)$ it suffices to: given $a : A$ give $c_0 : C(a, a, \text{refl}_a)$
- ▶ can deduce basic properties:
 - ▷ $\text{inv} : (a = b) \rightarrow (b = a)$ defined by $\text{inv}(\text{refl}_a) := \text{refl}_a$
 - ▷ $\cdot : (a = b) \rightarrow (b = c) \rightarrow (a = c)$ defined by $\text{refl}_a \cdot \text{refl}_a := \text{refl}_a$
 - ▷ $\text{ap}_f : (a = b) \rightarrow f(a) = f(b)$ defined by $\text{ap}_f(\text{refl}_a) := \text{refl}_{f(a)}$
- ▶ types representing propositions should have all term equal:

$$\text{isProp}(A) := \prod_{a:A} \prod_{b:A} a = b$$

- ▷ one can show $\text{isProp}(\text{isProp}(A))$

Identity = isomorphism?

- ▶ Problem: impossible to prove $\text{isProp}(a = b)$

Identity = isomorphism?

- ▶ Problem: impossible to prove $\text{isProp}(a = b)$
- ▶ Could add axiom to force this true (e.g. Lean4)

Identity = isomorphism?

- ▶ Problem: impossible to prove $\text{isProp}(a = b)$
- ▶ Could add axiom to force this true (e.g. Lean4)
- ▶ Voevodsky: embrace this!
 - ▷ think of terms $p : a = b$ as *isomorphisms* between $a = b$

Identity = isomorphism?

- ▶ Problem: impossible to prove $\text{isProp}(a = b)$
- ▶ Could add axiom to force this true (e.g. Lean4)
- ▶ Voevodsky: embrace this!
 - ▷ think of terms $p : a = b$ as *isomorphisms* between $a = b$
- ▶ What is $A = B$ for $A, B : \mathbf{Type} \rightsquigarrow$ *Univalence Axiom*

Identity = isomorphism?

- ▶ Problem: impossible to prove $\text{isProp}(a = b)$
- ▶ Could add axiom to force this true (e.g. Lean4)
- ▶ Voevodsky: embrace this!
 - ▷ think of terms $p : a = b$ as *isomorphisms* between $a = b$
- ▶ What is $A = B$ for $A, B : \mathbf{Type} \rightsquigarrow$ *Univalence Axiom*
- ▶ Can for $f : A \rightarrow B$ define a proposition $\text{isequiv}(f)$

$$A \simeq B := \sum_{f:A \rightarrow B} \text{isequiv}(f) \quad \text{idtoequiv}(A, B) : (A = B) \rightarrow (A \simeq B)$$

Identity = isomorphism?

- ▶ Problem: impossible to prove $\text{isProp}(a = b)$
- ▶ Could add axiom to force this true (e.g. Lean4)
- ▶ Voevodsky: embrace this!
 - ▷ think of terms $p : a = b$ as *isomorphisms* between $a = b$
- ▶ What is $A = B$ for $A, B : \mathbf{Type} \rightsquigarrow$ *Univalence Axiom*
- ▶ Can for $f : A \rightarrow B$ define a proposition $\text{isequiv}(f)$

$$A \simeq B := \sum_{f:A \rightarrow B} \text{isequiv}(f) \quad \text{idtoequiv}(A, B) : (A = B) \rightarrow (A \simeq B)$$

$$\overline{\text{univalence}(A, B) : \text{isequiv}(\text{idtoequiv}(A, B))}$$

- ▶ Roughly speaking $(A = B) \simeq (A \simeq B)$

Consequences of univalence

$$\text{isSet}(A) := \prod_{a,b:A} \text{isProp}(a = b)$$

► For A, B sets $(A = B) \simeq (\text{bijection } A \rightarrow B)$

Consequences of univalence

$$\text{isSet}(A) := \prod_{a,b:A} \text{isProp}(a = b)$$

- ▶ For A, B sets $(A = B) \simeq (\text{bijection } A \rightarrow B)$
- ▶ Define groups as *sets* with operations & usual relations, then for $G, H : \text{Group}$

$$(G = H) \simeq (\text{group isomorphism } G \rightarrow H)$$

- ▷ Any construction on groups is invariant under isomorphism!

Consequences of univalence

$$\text{isSet}(A) := \prod_{a,b:A} \text{isProp}(a = b)$$

- ▶ For A, B sets $(A = B) \simeq (\text{bijection } A \rightarrow B)$
- ▶ Define groups as *sets* with operations & usual relations, then for $G, H : \text{Group}$

$$(G = H) \simeq (\text{group isomorphism } G \rightarrow H)$$

- ▶ Any construction on groups is invariant under isomorphism!
- ▶ In category theory (once defined correctly):
 - ▶ Representatives of functors are unique
 - ▶ Adjoints are unique
 - ▶ Equivalence of categories = essentially surjective + fully faithful (no AC)

Models: Topological spaces

► Model:

▷ types $A, B, \dots \leftrightarrow$ (nice) spaces (e.g. CW complexes) $X, Y, \dots$

Models: Topological spaces

► Model:

- ▷ types $A, B, \dots \leftrightarrow$ (nice) spaces (e.g. CW complexes) $X, Y, \dots$
- ▷ terms $a : A \leftrightarrow$ points $x \in X$

Models: Topological spaces

► Model:

- ▷ types $A, B, \dots \leftrightarrow$ (nice) spaces (e.g. CW complexes) $X, Y, \dots$
- ▷ terms $a : A \leftrightarrow$ points $x \in X$
- ▷ functions $f : A \rightarrow B \leftrightarrow$ continuous maps $f : X \rightarrow Y$

Models: Topological spaces

► Model:

- ▷ types $A, B, \dots \leftrightarrow$ (nice) spaces (e.g. CW complexes) $X, Y, \dots$
- ▷ terms $a : A \leftrightarrow$ points $x \in X$
- ▷ functions $f : A \rightarrow B \leftrightarrow$ continuous maps $f : X \rightarrow Y$
- ▷ identities $p : a = b \leftrightarrow$ paths $x \rightsquigarrow y$

Models: Topological spaces

► Model:

- ▷ types $A, B, \dots \leftrightarrow$ (nice) spaces (e.g. CW complexes) $X, Y, \dots$
- ▷ terms $a : A \leftrightarrow$ points $x \in X$
- ▷ functions $f : A \rightarrow B \leftrightarrow$ continuous maps $f : X \rightarrow Y$
- ▷ identities $p : a = b \leftrightarrow$ paths $x \rightsquigarrow y$
- ▷ type families $\leftrightarrow$ fibrations

Models: Topological spaces

- ▶ Model:
 - ▷ types $A, B, \dots \leftrightarrow$ (nice) spaces (e.g. CW complexes) $X, Y, \dots$
 - ▷ terms $a : A \leftrightarrow$ points $x \in X$
 - ▷ functions $f : A \rightarrow B \leftrightarrow$ continuous maps $f : X \rightarrow Y$
 - ▷ identities $p : a = b \leftrightarrow$ paths $x \rightsquigarrow y$
 - ▷ type families $\leftrightarrow$ fibrations
- ▶ Can prove statements in Homotopy theory using proofs in HoTT
 - ▷ Blakers-Massey Theorem, $\pi_3(S^2) = \mathbb{Z}, \dots$

Models: Topological spaces

- ▶ Model:
 - ▷ types $A, B, \dots \leftrightarrow$ (nice) spaces (e.g. CW complexes) $X, Y, \dots$
 - ▷ terms $a : A \leftrightarrow$ points $x \in X$
 - ▷ functions $f : A \rightarrow B \leftrightarrow$ continuous maps $f : X \rightarrow Y$
 - ▷ identities $p : a = b \leftrightarrow$ paths $x \rightsquigarrow y$
 - ▷ type families $\leftrightarrow$ fibrations
- ▶ Can prove statements in Homotopy theory using proofs in HoTT
 - ▷ Blakers-Massey Theorem, $\pi_3(S^2) = \mathbb{Z}, \dots$
 - ▷ computer verifiable
 - ▷ no topological spaces necessary!

Models: ∞ -topoi

- ▶ Types form an ∞ -category **Type**

Models: ∞ -topoi

- Types form an ∞ -category **Type**

Definition (sketch)

An ∞ -category C consists of

1. a type $\text{ob}(C)$ of *objects*
2. for objects X, Y a type $\text{Hom}(X, Y)$ of *morphisms* from X to Y
3. identity morphisms, composition, associativity, whiskering, etc. (?!?!)

Models: ∞ -topoi

- Types form an ∞ -category **Type**

Definition (sketch)

An ∞ -category C consists of

1. a type $\text{ob}(C)$ of *objects*
2. for objects X, Y a type $\text{Hom}(X, Y)$ of *morphisms* from X to Y
3. identity morphisms, composition, associativity, whiskering, etc. (?!?!)

An ∞ -topos is an ∞ -category which behaves like **Type** for the purposes of colimits and finite limits.

- Homotopy Type Theory can be realized in any ∞ -topos.

Examples of ∞ -topoi

- ▶ $\mathrm{Sh}_{\mathrm{zar}}((\mathrm{CAlg}_k^{\mathrm{f.p.}})^{\mathrm{op}})$.
 - ▷ internally $R : \mathrm{Ring}$

Examples of ∞ -topoi

- ▶ $\mathrm{Sh}_{\mathrm{zar}}((\mathrm{CAlg}_k^{\mathrm{f.p.}})^{\mathrm{op}})$.
 - ▷ internally $R : \mathrm{Ring}$
 - ▷ have (see [1])

$$\mathbb{P}^1 \simeq \sum_{L \subseteq R^{n+1} \text{ submodule}} \|L \simeq R\|_{-1}$$

Examples of ∞ -topoi

- ▶ $\mathrm{Sh}_{\mathrm{zar}}((\mathrm{CAlg}_k^{\mathrm{f.p.}})^{\mathrm{op}})$.
 - ▷ internally $R : \mathrm{Ring}$
 - ▷ have (see [1])

$$\mathbb{P}^1 \simeq \sum_{L \subseteq R^{n+1} \text{ submodule}} \|L \simeq R\|_{-1}$$

- ▶ $\mathrm{Sh}_{\mathrm{at}}(\text{Borel probability spaces}) = \{\text{probability sheaves}\}$
 - ▷ see [5] and [4]

Examples of ∞ -topoi (simplicial type theory)

- ▶ Idea by [3]: $\mathbf{Cat}_\infty \hookrightarrow \mathbf{Psh}(\Delta) = \mathbf{Fun}(\Delta^{\mathrm{op}}, \mathbf{Type})$.

Examples of ∞ -topoi (simplicial type theory)

- ▶ Idea by [3]: $\text{Cat}_\infty \hookrightarrow \text{Psh}(\Delta) = \text{Fun}(\Delta^{\text{op}}, \mathbf{Type})$.
- ▶ Get type $\mathbb{I}$ corresponding to $\bullet \rightarrow \bullet$, and many axioms (exactly which is active research)

Examples of ∞ -topoi (simplicial type theory)

- ▶ Idea by [3]: $\text{Cat}_\infty \hookrightarrow \text{Psh}(\Delta) = \text{Fun}(\Delta^{\text{op}}, \mathbf{Type})$.
- ▶ Get type $\mathbb{I}$ corresponding to $\bullet \rightarrow \bullet$, and many axioms (exactly which is active research)
- ▶ (following [2]) have types corresponding to the following simplicial sets:

$$\Delta^2 = \begin{array}{ccc} \bullet & \xrightarrow{\quad} & \bullet \\ & \searrow & \downarrow \\ & & \bullet \end{array} \quad \Lambda_1^2 = \begin{array}{ccc} \bullet & \xrightarrow{\quad} & \bullet \\ & & \downarrow \\ & & \bullet \end{array} \quad E = \bullet \begin{array}{c} \xrightarrow{\quad} \\ \xleftarrow{\quad} \end{array} \bullet$$

Examples of ∞ -topoi (simplicial type theory)

- ▶ Idea by [3]: $\text{Cat}_\infty \hookrightarrow \text{Psh}(\Delta) = \text{Fun}(\Delta^{\text{op}}, \mathbf{Type})$.
- ▶ Get type $\mathbb{I}$ corresponding to $\bullet \rightarrow \bullet$, and many axioms (exactly which is active research)
- ▶ (following [2]) have types corresponding to the following simplicial sets:

$$\Delta^2 = \begin{array}{ccc} \bullet & \xrightarrow{\quad} & \bullet \\ & \searrow & \downarrow \\ & & \bullet \end{array} \quad \Lambda_1^2 = \begin{array}{ccc} \bullet & \xrightarrow{\quad} & \bullet \\ & & \downarrow \\ & & \bullet \end{array} \quad E = \bullet \begin{array}{c} \xrightarrow{\quad} \\ \xleftarrow{\quad} \end{array} \bullet$$

$$\begin{aligned} \text{isCat}(C) := & \text{isequiv}((\Delta^2 \rightarrow C) \rightarrow (\Lambda_1^2 \rightarrow C)) \\ & \times \text{isequiv}(C \rightarrow (E \rightarrow C)) \end{aligned}$$

Examples of ∞ -topoi (simplicial type theory)

- ▶ Idea by [3]: $\text{Cat}_\infty \hookrightarrow \text{Psh}(\Delta) = \text{Fun}(\Delta^{\text{op}}, \mathbf{Type})$.
- ▶ Get type $\mathbb{I}$ corresponding to $\bullet \rightarrow \bullet$, and many axioms (exactly which is active research)
- ▶ (following [2]) have types corresponding to the following simplicial sets:

$$\Delta^2 = \begin{array}{ccc} \bullet & \xrightarrow{\quad} & \bullet \\ & \searrow & \downarrow \\ & & \bullet \end{array} \quad \Lambda_1^2 = \begin{array}{ccc} \bullet & \xrightarrow{\quad} & \bullet \\ & & \downarrow \\ & & \bullet \end{array} \quad E = \bullet \begin{array}{c} \xrightarrow{\quad} \\ \xleftarrow{\quad} \end{array} \bullet$$

$$\begin{aligned} \text{isCat}(C) := & \text{isequiv}((\Delta^2 \rightarrow C) \rightarrow (\Lambda_1^2 \rightarrow C)) \\ & \times \text{isequiv}(C \rightarrow (E \rightarrow C)) \end{aligned}$$

- ▶ Any function between categories is a functor!

Thanks for listening!

References

- [1] Felix Cherubini, Thierry Coquand, and Matthias Hutzler. “A foundation for synthetic algebraic geometry”. In: *Mathematical Structures in Computer Science* 34.9 (2024), pp. 1008–1053. DOI: 10.1017/S0960129524000239.
- [2] Daniel Gratzer. “The Yoneda embedding in simplicial type theory”. Slides of presentation at HOTT/UF 2025. 2025. URL: <https://hott-uf.github.io/2025/>.
- [3] Emily Riehl and Michael Shulman. *A type theory for synthetic ∞ -categories*. 2023. arXiv: 1705.07442 [math.CT]. URL: <https://arxiv.org/abs/1705.07442>.
- [4] Alex Simpson. “Equivalence and Conditional Independence in Atomic Sheaf Logic”. In: *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science*. LICS '24. ACM, July 2024, pp. 1–14. DOI: 10.1145/3661814.3662132. URL: <http://dx.doi.org/10.1145/3661814.3662132>.
- [5] Alex Simpson. “Three Toposes for Probability & Randomness”. Slides of at Topos HOTT and its applications Institute Colloquium June 6th 2024. 2024. URL: <https://>